

LegalEase

AI-Powered Legal Document Generator

Phase-Wise Technical Documentation

FastAPI + Streamlit + Google Gemini 3.8 Flash

Project Type	Generative AI / Full-Stack Application
Backend	FastAPI
Frontend	Streamlit
AI Model	Gemini 3.8 Flash
Document Formats	TXT, DOCX, PDF

Documentation basis: This report follows the phase/milestone structure of the supplied LegalEase project document and updates the implementation details to match the current project built in VS Code. The screenshots included are the screenshots supplied during development.

Table of Contents

No.	Section
1	Project Overview
2	Objectives, Scope and Expected Outcome
3	Prerequisites and Development Environment
4	System Architecture
5	Project Structure
6	Phase 1 – Model Selection and Architecture
7	Phase 1 – Environment Setup
8	Phase 2 – Core Functionalities
9	Phase 2 – Document Formatting and Export
10	Phase 3 – FastAPI Backend and API Logic
11	Phase 3 – Gemini AI Integration
12	Phase 4 – Streamlit Frontend
13	Phase 4 – Preview, Editing and Downloads
14	Phase 5 – Local Deployment
15	Phase 5 – Testing and Verification
16	Phase 6 – End-to-End Workflow
17	User Input Guide
18	API Reference
19	Security, Reliability and Limitations
20	Development Screenshots
21	Application Screenshots and Output Evidence
22	Future Enhancements
23	Conclusion

1. Project Overview

LegalEase is a Generative AI based legal-document drafting application. The system accepts structured information such as document type, parties, terms and an effective date, sends those requirements to Google Gemini, and presents the resulting draft in a web interface. The current implementation separates the user interface, API layer, AI generation layer and document export layer.

Project Description

- Generate structured legal-document drafts from user-provided facts.
- Provide an editable document area so the user can review or modify generated text.
- Render a readable preview before download.
- Export the final text as TXT, DOCX or PDF.
- Keep the Gemini API key outside the source code through environment variables.
- Provide FastAPI endpoints that can be tested independently from the Streamlit interface.

Typical Scenarios

- A startup founder prepares an employment-related draft from role, party and compensation information.
- A freelancer prepares an NDA by entering the parties, confidentiality requirements and effective date.
- A landlord prepares a lease draft by supplying tenant, property and lease terms.

Important Use Limitation

LegalEase is a drafting aid, not a substitute for legal advice. Generated content can contain omissions or errors and should be reviewed by a qualified legal professional before actual use.

2. Objectives, Scope and Expected Outcome

Objectives

- Create a simple interface for collecting legal-document requirements.
- Use Gemini 3.8 Flash to transform those requirements into a structured draft.
- Maintain a modular backend so generation and export logic are separated.
- Support multiple document output formats.
- Provide validation and controlled handling of user input.
- Make the project easy to run locally and suitable for future cloud deployment.

Scope

- Current scope covers document drafting, preview, editing and export.
- The current system does not perform authoritative legal research or guarantee jurisdiction-specific legal validity.
- The current application is configured for local development and can be containerized for deployment.

Expected Outcome

The completed application should allow a user to enter document details, click Generate Document, receive an AI-generated draft, review or edit it, and save the resulting content in the supported formats.

3. Prerequisites and Development Environment

Software Prerequisites

- Python 3.10 or later is recommended for the project environment.
- VS Code or another Python-capable IDE.
- A Google Gemini API key.
- Internet access for Gemini API requests.
- Optional: Docker and Docker Compose for containerized execution.

Main Dependencies

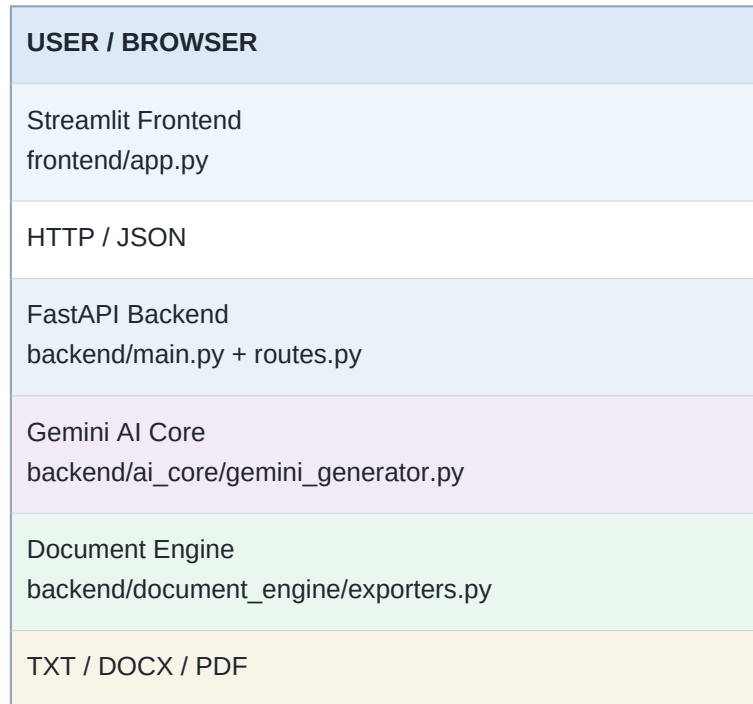
Package	Purpose
fastapi	Backend API framework
uvicorn	ASGI development server
streamlit	Web-based frontend
google-genai	Gemini API client
pydantic	Request/response validation
python-docx	DOCX generation
fpdf2	PDF generation
requests	Frontend-to-backend HTTP calls
python-dotenv	Environment variable loading
pytest	Automated tests

Environment Variables

The project uses a .env file locally. The important variables are GOOGLE_API_KEY, GEMINI_MODEL and BACKEND_URL. The .env file should never be committed to a public repository.

4. System Architecture

The current application uses a layered architecture. Streamlit handles user interaction, FastAPI exposes the application API, the Gemini integration module generates document text, and the document engine converts the text into downloadable formats.



Request Flow

- The user enters document details in Streamlit.
- The frontend sends JSON to POST /generate.
- FastAPI validates the payload using Pydantic.
- GeminiDocumentGenerator builds a structured prompt and calls Gemini 3.8 Flash.
- The generated text is returned to the frontend.
- The user reviews or edits the text.
- Export requests are sent to the corresponding backend endpoint.

5. Project Structure

```
LegalEase/  
├── backend/  
│   ├── main.py  
│   ├── models.py  
│   ├── routes.py  
│   └── ai_core/  
│       ├── gemini_generator.py  
│       ├── document_engine/  
│       │   └── exporters.py  
│       └── utils/  
│           └── sanitizer.py  
├── frontend/  
│   └── app.py  
├── tests/  
│   ├── test_api.py  
│   └── test_documents.py  
├── assets/  
├── .vscode/  
├── .env  
├── .env.example  
├── requirements.txt  
├── Dockerfile  
├── docker-compose.yml  
├── Procfile  
└── README.md
```

Module Responsibilities

- **main.py:** Creates the FastAPI application, middleware and health/root endpoints.
- **models.py:** Defines validated request and response schemas.
- **routes.py:** Implements generation and export endpoints.
- **gemini_generator.py:** Loads the API key, builds the prompt and calls Gemini.
- **exporters.py:** Creates TXT, DOCX and PDF output.
- **sanitizer.py:** Normalizes generated text and safely prepares preview content.
- **frontend/app.py:** Provides the Streamlit user experience.
- **tests/:** Contains automated API and document-export tests.

6. Phase 1 – Model Selection and Architecture

Activity 1.1: Select the Generative AI Model

The supplied project reference describes Gemini as the AI foundation for legal-document generation. The current implementation updates the model configuration to **gemini-3.8-flash**, stored in GEMINI_MODEL. The application does not hard-code the API key and allows the model name to be changed through the environment.

Why the model is used in LegalEase

- It accepts a structured prompt containing document type, parties, date and terms.
- It returns natural-language document content that can be rendered and exported.
- The prompt explicitly instructs the model not to invent material facts.
- Missing material information is represented using placeholders.

Activity 1.2: Define the Architecture

The architecture follows the milestone structure in the supplied reference: Streamlit for interaction, FastAPI for API handling, a dedicated Gemini integration module for generation, and formatting modules for document export.

7. Phase 1 – Environment Setup

Activity 1.3: Development Environment

- Create a Python virtual environment named venv.
- Activate the environment before installing or running the project.
- Install dependencies from requirements.txt.
- Create a local .env file with the Gemini API key and application configuration.
- Keep .env excluded through .gitignore.

Backend startup command

```
python -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend startup command

```
python -m streamlit run frontend/app.py
```

Verification endpoints

- Backend root: <http://127.0.0.1:8000/>
- Health check: <http://127.0.0.1:8000/health>
- Interactive API documentation: <http://127.0.0.1:8000/docs>
- Frontend: <http://localhost:8501>

8. Phase 2 – Core Functionalities

Activity 2.1: Legal Document Generation

The core function accepts four pieces of information: `document_type`, `parties`, `terms` and `effective_date`. The backend validates these fields and passes them to the Gemini generator.

Prompt Design

The generation prompt establishes LegalEase as a drafting assistant and gives the model explicit constraints. It requests a clear title, numbered sections, descriptive headings, supplied parties and effective date, relevant signature blocks, and an AI-generated-draft notice. It also prohibits unsupported legal citations and invented material facts.

Activity 2.2: Input Validation

- Pydantic models validate string lengths before generation.
- `DocumentRequest` limits document type to 100 characters, parties to 10,000 characters and terms to 20,000 characters.
- `ExportRequest` validates document type and generated content before creating a file.

9. Phase 2 – Document Formatting and Export

TXT Export

TXT export uses the sanitized generated content and returns UTF-8 encoded plain text with a download response.

DOCX Export

- Uses python-docx.
- Creates margins suitable for a formal document.
- Adds the document type as a centered title.
- Uses Times New Roman for generated document text.
- Adds LegalEase branding and a review footer.
- Optionally embeds assets/logo.png when the file exists.

PDF Export

- Uses FPDF2.
- Adds a centered document-type header.
- Adds an AI-generated-draft footer.
- Uses automatic page breaks for longer documents.
- Normalizes text before writing it into the PDF.

Sanitization

The sanitizer converts common typographic quotes, dashes, bullets and non-breaking spaces to safer equivalents and removes control characters. HTML preview content is escaped before being placed in the browser.

10. Phase 3 – FastAPI Backend and API Logic

Activity 3.1: Configure main.py

- Initialize the FastAPI application.
- Configure CORS for the local Streamlit frontend.
- Expose the root endpoint with project status information.
- Expose /health for a simple health check.
- Include the router from routes.py.

Activity 3.2: Implement routes.py

- Create a GeminiDocumentGenerator instance when a generation request arrives.
- Call generate_document with the validated fields.
- Return a DocumentResponse containing document_type and content.
- Return TXT, DOCX and PDF as HTTP download responses.
- Translate generation failures into an HTTP error that the frontend can display.

API Contract

Method	Endpoint	Purpose
GET	/	Service status
GET	/health	Health check
POST	/generate	Generate document
POST	/export/txt	Download TXT
POST	/export/docx	Download DOCX
POST	/export/pdf	Download PDF

11. Phase 3 – Gemini AI Integration

GeminiDocumentGenerator

The Gemini integration is isolated in `backend/ai_core/gemini_generator.py`. This separation makes it possible to change prompt logic, model configuration or retry behavior without changing the API routes.

Generation sequence

- Load `GOOGLE_API_KEY` from the environment.
- Load `GEMINI_MODEL` or use the configured default.
- Create a Google GenAI client.
- Build a structured prompt from the four user inputs.
- Call `client.models.generate_content(...)`.
- Read the returned text.
- Reject an empty response.
- Return the cleaned document text to the FastAPI route.

Temporary Service Failures

During development, the supplied screenshots show a Gemini 503 UNAVAILABLE response indicating temporary model capacity/high demand. The project therefore includes retry handling for temporary 503/429-style failures. Such a response is an upstream service condition and is distinct from a local FastAPI validation failure.

12. Phase 4 – Streamlit Frontend

Activity 4.1: User Interface

- Configure the Streamlit page.
- Display LegalEase branding and workflow guidance.
- Provide a document-type selector.
- Provide an effective-date field.
- Provide a parties text area.
- Provide a terms and conditions text area.
- Provide a Generate Document button.
- Show generated content in an editable area after successful generation.

Frontend-to-Backend Communication

The frontend reads `BACKEND_URL` from the environment and sends a JSON POST request to `/generate`. The request timeout is configured to allow time for an AI generation request. The returned document is stored in the Streamlit session and used for preview/export.

User Experience Goal

The interface follows a simple workflow: enter details → generate → review/edit → preview → download. This mirrors the workflow described in the supplied reference document while using the current implementation.

13. Phase 4 – Preview, Editing and Downloads

Dynamic Preview

The application uses a sanitized HTML preview to display generated text in a scrollable, document-like area. HTML escaping is applied so generated text is rendered as content rather than executable markup.

Editing

The generated document can be placed into a text editor area, allowing the user to modify wording before exporting.

Download Workflow

- TXT uses the current edited content.
- DOCX uses the current edited content and document type.
- PDF uses the current edited content and document type.
- The backend creates the requested file and returns it as an HTTP response.

Output Quality Controls

- Text sanitization before export.
- Formal title and typography in DOCX.
- Header/footer handling in PDF.
- UTF-8 encoding for TXT.
- Automatic page breaks for PDF output.

14. Phase 5 – Local Deployment

Activity 5.1: Prepare the Application

- Create and activate the virtual environment.
- Install requirements.txt.
- Configure .env.
- Start FastAPI on port 8000.
- Start Streamlit on port 8501.
- Verify /health before testing generation.

Activity 5.2: Deployment Assets

- Dockerfile is provided for containerization.
- docker-compose.yml is provided for Compose-based execution.
- Procfile is provided as a deployment-oriented process definition.
- requirements.txt records the Python dependencies.
- .vscode settings and launch configuration support local development.

Recommended Local Verification Order

1) Activate venv → 2) install dependencies → 3) configure .env → 4) start FastAPI → 5) verify /health → 6) start Streamlit → 7) enter test data → 8) generate → 9) review/edit → 10) export.

15. Phase 5 – Testing and Verification

Automated Tests

The project includes tests/test_api.py and tests/test_documents.py. These are intended to verify the API behavior and document-export functions.

Test Command

```
pytest -v
```

Manual Verification Checklist

Check	Expected result
FastAPI root	Returns LegalEase service information
/health	Returns status=ok
Generate request	Returns generated document text when Gemini is available
Preview	Displays sanitized document content
Edit	Allows document text to be modified
TXT	Downloads UTF-8 plain text
DOCX	Downloads a valid Word document
PDF	Downloads a valid PDF
Invalid input	Validation prevents malformed requests
Temporary Gemini failure	User receives a clear error/retry message

16. Phase 6 – End-to-End Workflow

Step-by-Step Execution

1. Open the Streamlit application in the browser.
2. Select the legal document type.
3. Enter the effective date.
4. Enter the parties and their roles.
5. Enter the terms and conditions.
6. Click Generate Document.
7. The frontend sends the request to FastAPI.
8. FastAPI validates the request and calls the Gemini integration.
9. Gemini returns the draft text.
10. The frontend displays the generated content.
11. Review and edit the document.
12. Export the final text as TXT, DOCX or PDF.

Example Test Data Used During Development

Field	Example
Document type	Non-Disclosure Agreement (NDA)
Effective date	October 1, 2026
Parties	John Doe - Consultant; ABC Technologies Ltd - Client
Terms	Confidential information remains private; project-only use; no third-party sharing; 30-day termination notice

17. User Input Guide

Document Type

Purpose: identifies the kind of legal draft to generate. Examples include NDA, employment contract, service agreement, lease agreement and other structured agreements.

Parties

Purpose: identifies the people or organizations involved and their roles. Example: John Doe – Consultant; ABC Technologies Ltd – Client.

Terms and Conditions

Purpose: describes the clauses the user wants incorporated. The current UI accepts free-form text, so each requirement can be written on its own line.

Effective Date

Purpose: identifies the date associated with the agreement. Example: October 1, 2026.

Good Input Practice

- Use precise party names and roles.
- State important commercial terms explicitly.
- Do not omit material dates or amounts when they are known.
- If a legal requirement is jurisdiction-specific, provide the required jurisdiction information rather than expecting the AI to infer it.

18. API Reference

POST /generate

Request body:

```
{
  "document_type": "Non-Disclosure Agreement",
  "parties": "John Doe - Consultant\nABC Technologies Ltd - Client",
  "terms": "Confidential information must remain private.",
  "effective_date": "October 1, 2026"
}
```

Response shape:

```
{
  "document_type": "Non-Disclosure Agreement",
  "content": "Generated legal document text..."
}
```

Export endpoints

- **POST /export/txt:** accepts document type and content and returns a text file.
- **POST /export/docx:** creates a Word document.
- **POST /export/pdf:** creates a PDF document.

FastAPI Swagger

```
http://127.0.0.1:8000/docs
```

19. Security, Reliability and Limitations

API Key Security

- The Gemini API key is read from `GOOGLE_API_KEY`.
- The `.env` file should remain local and be excluded from Git.
- Never paste an active API key into source code or public documentation.
- If a key is exposed, rotate/revoke it and update the local environment.

Input and Output Safety

- Pydantic validation limits request sizes.
- Generated text is sanitized before export.
- HTML preview escapes generated text.
- The AI prompt instructs the model not to invent material facts.

Reliability

Temporary Gemini 503/429 failures can occur independently of the application code. The implementation includes retry handling for temporary service errors. The user should still be prepared to retry later when the upstream service is unavailable.

Limitations

- Generated legal text is not guaranteed to be legally valid in a particular jurisdiction.
- The system does not replace professional legal review.
- The current application does not include a legal-source retrieval layer.
- A successful API call depends on Gemini availability and the configured API key.

20. Development Screenshots

The following screenshots are included exactly from the development evidence supplied with the project. They document the VS Code setup, configuration, dependency/server work and implementation state.

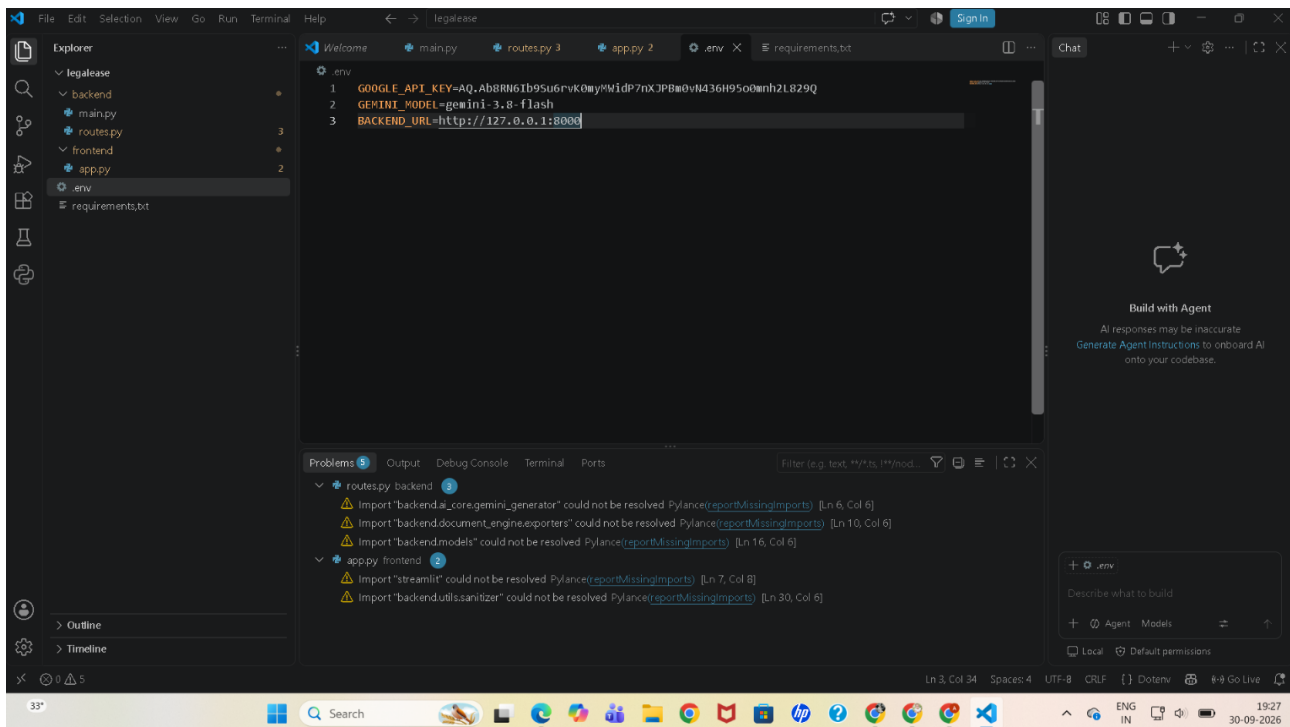


Figure 20.1 – VS Code project configuration and environment file during development.

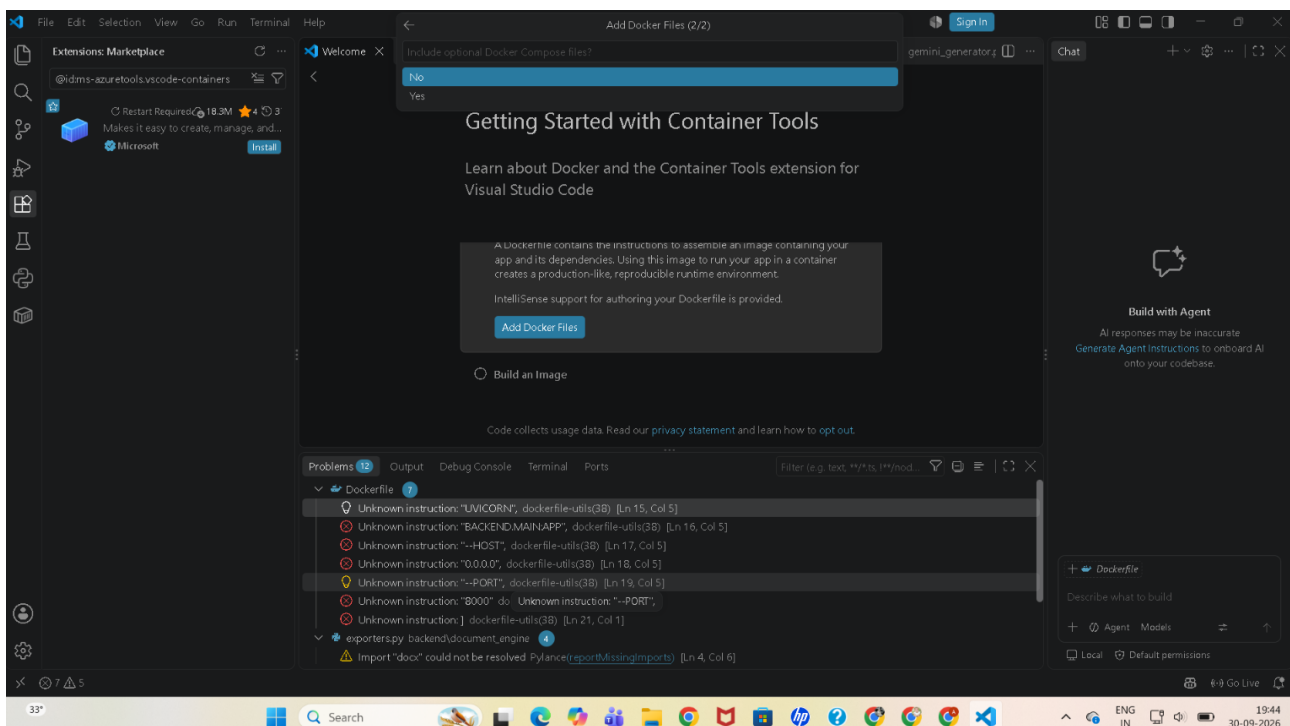


Figure 20.2 – VS Code development environment and project diagnostics.

20. Development Screenshots – Continued

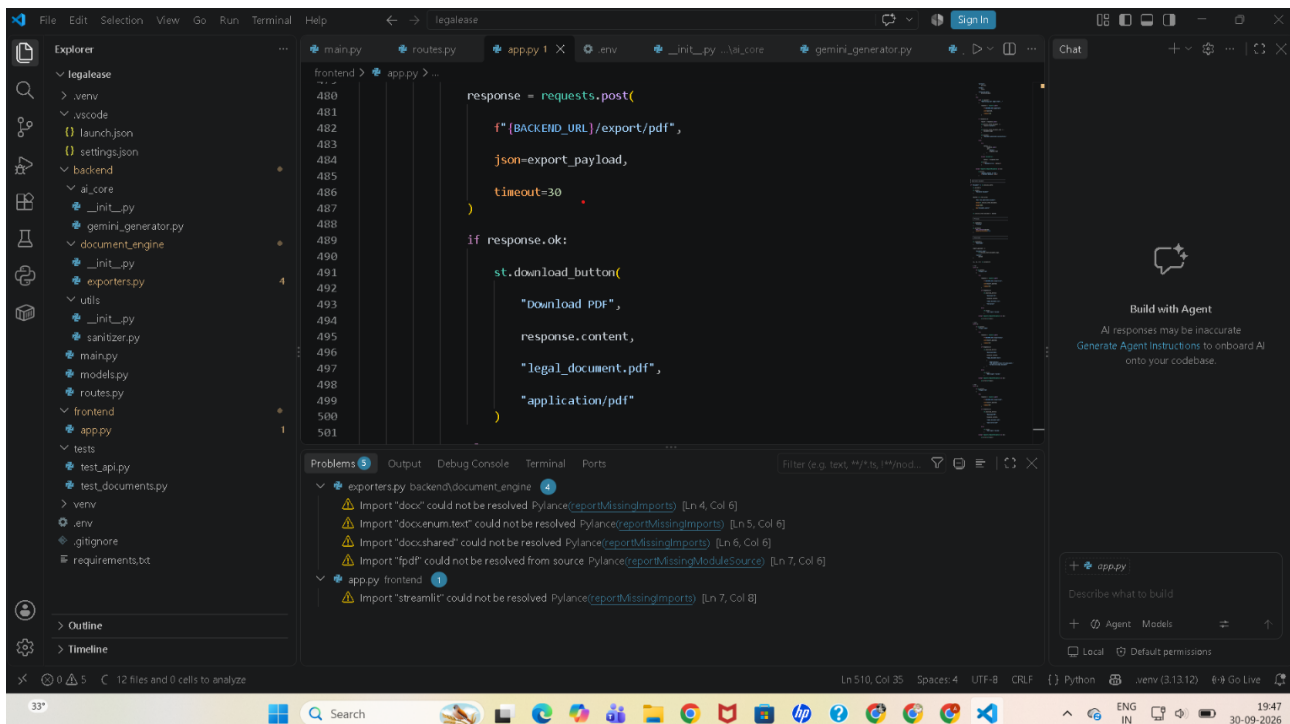


Figure 20.3 – Document export implementation visible in the VS Code editor.

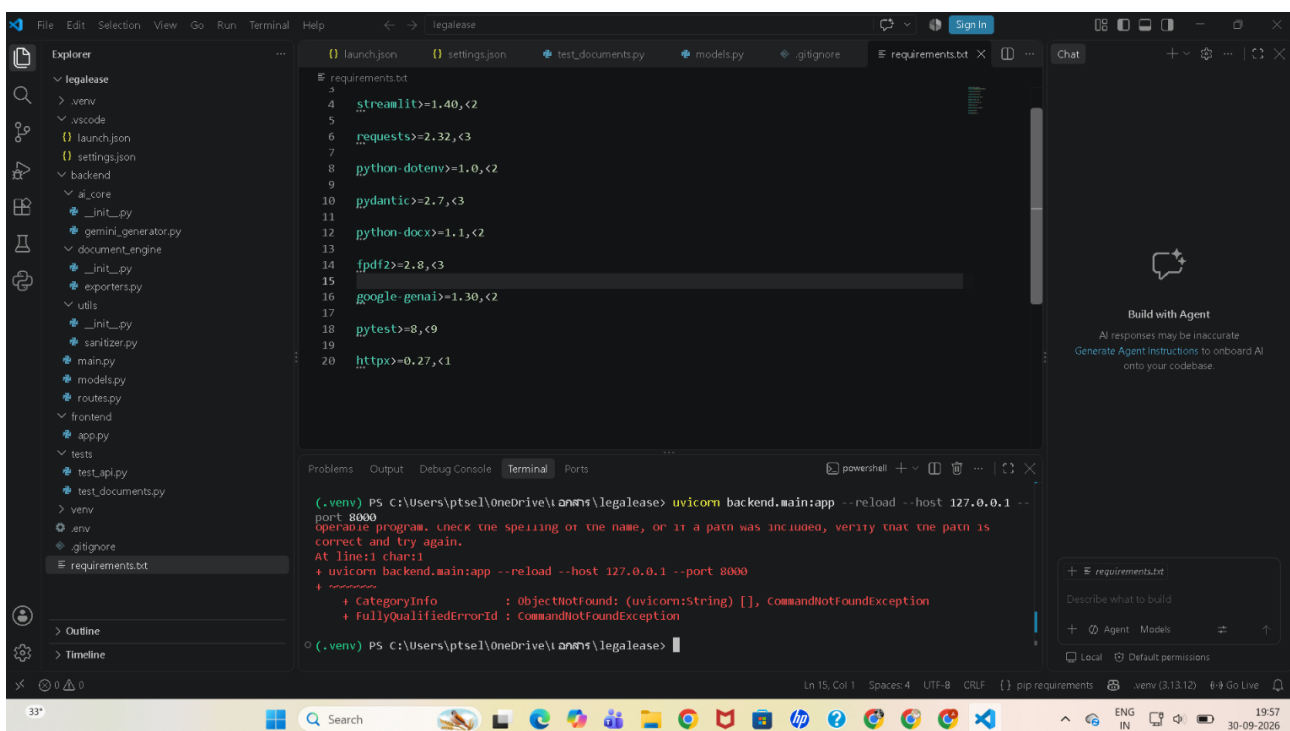


Figure 20.4 – Dependency/server setup during local development.

21. Application Screenshots and Output Evidence

The screenshots below show the actual Streamlit interface and the Gemini service response captured during development. The supplied evidence contains an application input screen and a 503 UNAVAILABLE response. No successful generated-document output screenshot was supplied, so this report does not represent the error screen as a successful generation result.

The screenshot shows the 'LegalEase' application running on a local host (localhost:8501). The interface has a sidebar on the left with the application name, a backend URL, and a workflow list. The main area is titled 'Document Details' and contains several input fields: 'Document Type' (set to 'Employment Contract'), 'Effective Date' (set to 'October 1, 2026'), 'Parties Involved' (a text box containing 'Jane Doe (Service Provider)' and 'TechNova Inc. (Client)'), and 'Terms & Conditions' (a text box containing 'Payment within 30 days of invoice; confidentiality must be maintained; either party may terminate with 15 days notice.'). A red 'Generate Document' button is at the bottom.

Figure 21.1 – LegalEase Streamlit document input interface with document type, date, parties and terms fields.

The screenshot shows the same 'LegalEase' application, but with different input data: 'Document Type' is 'Non-Disclosure Agreement (NDA)', 'Effective Date' is 'October 1, 2026', 'Parties Involved' is 'John Doe - Consultant' and 'ABC Technologies Ltd - Client', and 'Terms & Conditions' is 'All confidential information must remain private. Information may only be used for the project. Information must not be shared with third parties. Either party may terminate with 30 days notice.' The red 'Generate Document' button is still present. Below the button, a red error message box displays the following text: 'Backend error: Gemini request failed: 503 UNAVAILABLE. (error: {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'})'.

Figure 21.2 – LegalEase generation attempt showing the Gemini 503 UNAVAILABLE message returned by the upstream service.

22. Future Enhancements

- Add user authentication and role-based access.
- Add document history and version tracking.
- Add a reusable legal-template library.
- Add jurisdiction-aware workflows.
- Add retrieval-augmented generation using approved legal sources.
- Add richer DOCX/PDF styling and configurable branding.
- Add multilingual generation and translation workflows.
- Add cloud storage and organization/team support.
- Add stronger audit logging and privacy controls.
- Add automated end-to-end browser tests.
- Add better status messaging and a visible retry countdown for temporary Gemini capacity errors.

23. Conclusion

LegalEase demonstrates how Generative AI can be integrated into a practical document-drafting workflow. The application combines a Streamlit frontend, a FastAPI backend, a dedicated Gemini generation layer and document-export utilities. This separation keeps the system understandable and makes individual components easier to test and improve.

The current implementation supports the major workflow defined in the supplied project reference: collecting legal-document inputs, generating a structured draft, reviewing/editing the result and exporting it in multiple formats. The project also introduces environment-based API configuration, validation, sanitization and temporary-error handling.

The screenshots supplied during development provide evidence of the implemented interface and development environment. The recorded 503 Gemini response demonstrates why production-grade AI applications should treat upstream model availability as a separate operational concern.

Final note: LegalEase should be treated as an AI-assisted drafting system. Generated documents should be reviewed by a qualified legal professional before being relied upon or executed.

End of Documentation